

LAPORAN PRAKTIKUM TEKNOLOGI IoT II
Komunikasi Asinkron dan WebSockets



Oleh:

NAMA : AKBAR BETHARA IMANI

NPM : 25782088

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

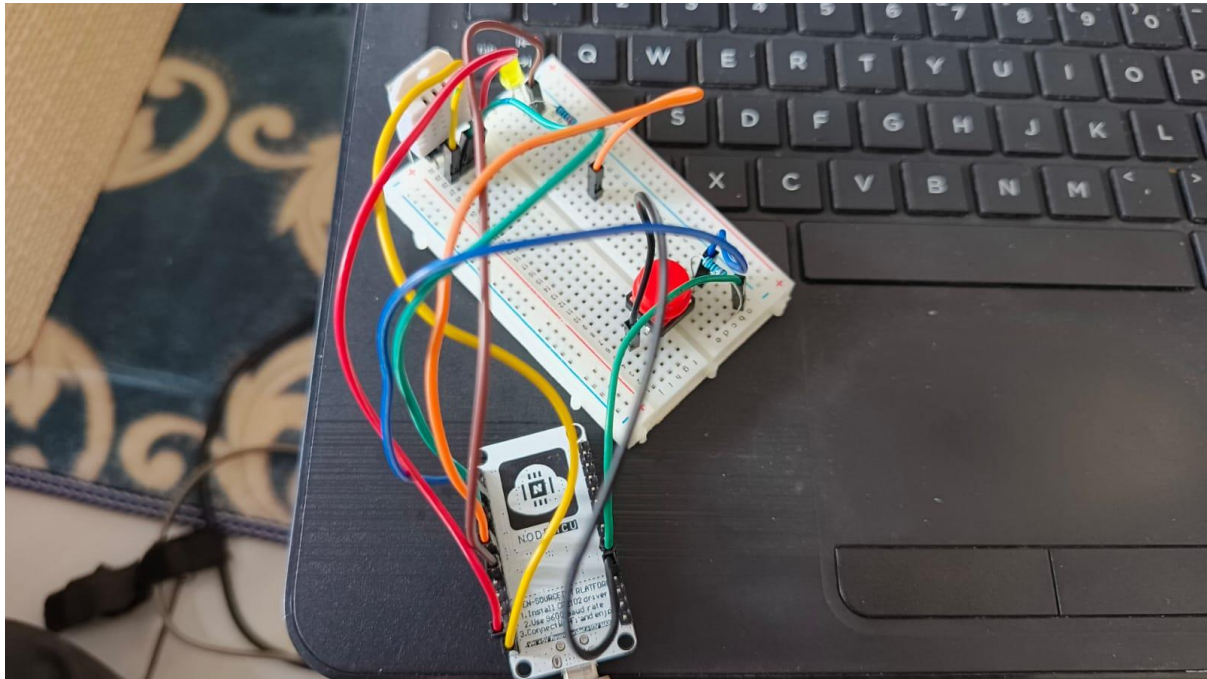
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026/2027

1. Rangkaian

Komponen	Koneksi
LED (anoda, kaki panjang)	D6 (GPIO 12)
LED (katoda, kaki pendek)	Resistor 220 Ohm, lalu GND
DHT22 (DATA)	D4 (GPIO 2); VCC ke 3V3; GND ke GND
Push button	Satu kaki ke 3V3; kaki seberang ke D2 (GPIO 4)
Resistor 10k (pull-down)	Dari kaki tombol di D2 ke GND



2. Cara Kerja Program

Program berjalan sebagai web server asinkron (port 80) dan WebSocket (rute /ws) di NodeMCU. Halaman HTML disimpan di memori program (PROGMEM). Setiap kali ada perubahan (klik tombol web, tombol fisik ditekan, atau data sensor baru), fungsi notifyClients() mengirim paket JSON ke semua browser yang terhubung. Pembacaan DHT22 dijalankan setiap 3 detik dengan millis() tanpa delay(), sehingga tombol fisik tetap responsif.

3. Hasil Pengujian

3.1 Mendapatkan IP Address

Setelah upload, IP address ditampilkan di Serial Monitor (115200 baud).

```
IP Address Server Anda:  
10.222.108.219
```

3.2 Uji 1: Kendali Web Tersinkronisasi

Halaman dibuka di browser laptop, lalu tombol diklik di laptop.

Hasil: led menyala bersamaan dengan tombol di klik

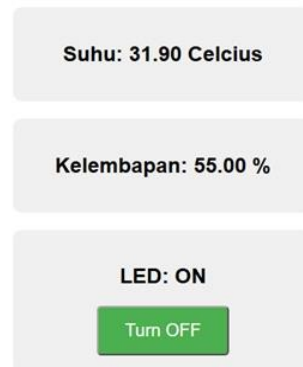
3.3 Uji 2: Kendali Perangkat Keras

Push button fisik ditekan.

Hasil: status led berubah dan led juga ikut berubah

3.4 Telemetry Suhu dan Kelembapan (Latihan 4)

Halaman menampilkan suhu dan kelembapan yang diperbarui otomatis setiap 3 detik.



4. Jawaban Latihan

Latihan 1: Mengapa WebSockets lebih hemat bandwidth dan tidak membuat layar berkedip dibanding Meta Refresh?

Pada Meta Refresh, browser harus meminta ulang seluruh halaman HTML setiap interval waktu tertentu. Setiap permintaan membawa header HTTP lengkap dan seluruh isi halaman, walaupun data yang berubah hanya satu angka. Karena halaman dimuat ulang, layar juga berkedip.

WebSockets membuka satu koneksi yang tetap terbuka (persistent) dan dua arah. Halaman HTML dimuat sekali saja. Setelah itu server hanya mengirim paket kecil (JSON berisi data yang berubah) dengan header yang sangat ringan, dan JavaScript hanya mengganti teks pada elemen yang perlu. Server juga dapat mengirim data sendiri (push) saat ada perubahan, tanpa menunggu browser bertanya. Hasilnya, data lebih hemat dan tampilan tidak berkedip.

Latihan 2: Bahaya delay(3000) di dalam void loop()

delay(3000) menghentikan seluruh eksekusi loop() selama 3 detik. Akibatnya:

- Tombol fisik tidak terbaca selama jeda, sehingga penekanan tombol bisa terlewat dan respon terasa lambat.
- Perintah dari web baru dieksekusi pada baris digitalWrite() di loop(), sehingga LED fisik terlambat berubah hingga 3 detik.
- Debounce yang bergantung pada millis() menjadi tidak akurat.

- Tugas internal seperti ws.cleanupClients() terlambat dijalankan, dan jeda yang panjang berisiko memicu watchdog reset pada ESP8266.

Karena itu digunakan teknik non-blocking dengan millis(): program hanya memeriksa selisih waktu dan terus berjalan, sehingga tombol, LED, dan WebSocket tetap responsif.

Latihan 3: ID elemen HTML yang diubah oleh blok "Merombak UI Tombol & Teks"

Blok tersebut mengubah dua elemen:

1. ledStatus (span id="ledStatus"): teksnya diganti menjadi "ON" atau "OFF".
2. toggleBtn (button id="toggleBtn"): teks tombol diganti ("Turn OFF" atau "Turn ON") dan kelas CSS-nya diganti (btn-on atau btn-off) sehingga warnanya berubah hijau atau merah.

Elemen tempValue diubah oleh blok terpisah di atasnya, yaitu untuk menampilkan data suhu.

Latihan 4: Pengembangan Telemetry Kelembapan

Modifikasi dilakukan pada tiga bagian:

- C++ (variabel): ditambahkan variabel penampung currentHum bertipe String dengan nilai awal "--".
- C++ (notifyClients): paket JSON ditambah pasangan key "hum" yang berisi currentHum, sehingga formatnya menjadi {"led":"...", "suhu":"...", "hum":"..."}
- C++ (loop): di dalam timer non-blocking millis() 3 detik, nilai kelembapan dibaca dengan dht.readHumidity(), disimpan ke currentHum jika valid (bukan NaN), lalu notifyClients() dipanggil.
- HTML: ditambahkan kartu baru dengan elemen span berid humValue untuk menampilkan kelembapan.
- JavaScript: pada fungsi onMessage(event) ditambahkan pengecekan dataObj.hum, lalu nilainya dimasukkan ke elemen humValue.

```
// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
String currentHum = "--";
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;
```

```
void notifyClients(){
  String jsonString="{\"led\": \"\"+String(ledState?.1::0)+\"\", \"";
  jsonString+="{\"suhu\": \"\"+currentTemp+\"\", \"";
  jsonString+="{\"hum\": \"\"+currentHum+\"\"}";
  ws.textAll(jsonString);
}
```

```
if ((millis() - lastTime) > 3000) {
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (!isnan(t)) currentTemp = String(t);
  if (!isnan(h)) currentHum = String(h);

  if (!isnan(t) || !isnan(h)) {
    notifyClients();
  }
  lastTime = millis();
}
```

